Cloud Computing Project

CivicLink

Group members:

22/ENG/017-B.M.A.U.RATHNAYAKE

22/ENG/078-K.A.P.M.PERERA

22/ENG/079-K.M.L.N.SENADHEERA

INTRODUCTION

In the present internet age, effective communication between citizens and local authorities is vital for maintaining and improving community infrastructure. **CivicLink** is a cloud-based civic issues reporting platform designed to simplify this process by allowing individuals to easily report concern such as damaged roads, utility faults, and environmental hazards. At the same time, it equips authorities with tools to track, manage, and resolve issues efficiently. By digitalizing civic engagement, CivicLink fosters more responsive governance and contributes to enhancing the quality of life within communities.

Objective

The primary objective of **CivicLink** is to develop a cross-platform flutter based mobile application that enables citizens to easily report community issues using features like photo capture, GPS-based locations detection, and push notifications. The app provides an intuitive mobile-first interface with offline functionality for low-connectivity areas and syncs data through a cloud backend. Citizens can quickly submit reports, while authorities manage them through their interface, ensuring efficiency, transparency, and stronger civic engagement.

Key Features

There are two user types in Civiclink

- Citizens
- Officials

Citizen features

- Report issues

  Citizens can report problems to the relevant department by selecting a category, assigning a priority level (low, medium, high, critical) and attaching a photo with a brief description. The app automatically detects the user's location to ensure accurate reporting

- Track issue status

  Citizens can monitor the progress of their reports, viewing statuses such as pending, in-progress, or resolved. They can also send reminders to departments if an issue remains unresolved.

- Issue map

  Citizens can explore issues reported within a chosen radius of their location. The search can be filtered by department and radius, with results displayed in either map or list view.

Official features

- Respond to issues

  Reported issues appear in the department dashboard, with urgent ones clearly highlighted. Officials can also track newly reported cases under the "New Today" section for quick attention.

- Issue assignment

  Pending issues are grouped under bulk actions, allowing officials to review, assign, and update them efficiently. Citizens are notified when their issue is assigned or updated.

- Updating issue status

  Officials can change the status of a report to in-progress, resolved, or rejected, ensuring transparency and timely feedback to citizens.

- Performance tracking

  The admin dashboard provides detailed analytics, including resolution rate, average response time, citizen satisfaction, issue distribution by status and priority, and emerging trends, helping departments monitor and improve their overall performance.

Common features for both citizens and officials

- User profile

  Both citizens and officials can create and manage personalized profiles within the app.

- Notifications

  Real-time notifications keep users updated on issue status changes, assignments, and other important updates.

- Settings & Preferences

  Users can customize their app experience by enabling dark mode, selecting preferred languages, and managing privacy and security options. They can also access help and support, send feedback, and rate the app.

Technical implementation

CivicLink is built using Flutter, Google's powerful cross-platform framework that allows us to deliver a single codebase for android while ensuring a seamless, responsive, and mobile-first experience. The app is supported by Firebase and Cloud Firestore, which handle authentication, database management, cloud storage, and hosting. This combination provides real-time synchronization, secure data handling, and scalability to meet the dynamic needs of both citizens and officials. To enhance location-based features, CivicLink integrates the Google Maps API, enabling automatic detection of user locations, mapping of reported issues, and location-based filtering for better transparency and accessibility.

- Cloud Firestore – Stores issue reports, user profiles, and media files with real-time synchronization, ensuring citizens and officials receive instant updates.
- Firebase Authentication – Provides secure login and role-based access for both citizens and officials.
- Firebase Cloud Messaging (FCM) – Delivers push notifications for issue status updates, reminders, and important alerts.
- Google Maps API – Powers location services, including automatic GPS detection, issue mapping, and radius-based search

REQUIREMENTS

Functional Requirements

Functional requirements define the core functionalities of CivicLink and how users (citizens and officials) interact with the system. They describe what the app should do under different scenarios to achieve its objectives.

User Management

- Registration and Authentication
    - Users must be able to register using email, password, or optional social login.
    - Firebase authentication will handle secure login, registration, and password recovery.
    - Users can update profile information such as name, contact details, and role.

- User Roles
    - The system distinguishes between Citizen and Official roles.
    - Citizens can report issues and track their status.
    - Official can manage, assign, and resolve reported issues through the dashboard.

Citizen Functionalities

- Report issues: Citizens can create reports by selecting a category, priority level, adding a description, and attaching images. The app automatically detects the user's location for accurate reporting.
- Track issue status: Citizens can view the status of their reports (pending, in-progress, resolved) and send reminders if needed.
- Issue Map: Citizens can explore reported issues within a specified radius, filter by department, and view results in map or list format.

Official Functionalities

- Respond to issues: Officials can view urgent issues and newly reported issues in the "New Today" section for quick action

- Assign issues: Officials can assign pending issues to themselves and notify citizens.
- Update issue status: Officials can change issue status to in-progress, resolved, or rejected.
- Performance tracking: Officials can monitor analytics, including resolution rate, average response time, citizen satisfaction, and trends.

Common Functionalities

- User Profile: Both roles can manage their profiles.
- Notifications: Real-time push notifications for issue updates, assignments, and reminders.
- Settings & Preferences: Users can enable dark mode, change language, manage privacy settings, access help, send feedback, and rate the app.

Non-Functional Requirements

Non-functional requirements define the quality attributes of CivicLink, focusing on performance, reliability, usability, and security. They describe how well the system should operate rather than specific features.

Performance

- The app should load within 3 seconds under standard internet conditions.
- The system must handle hundreds of simultaneous users and thousands of reports without performance degradation.

Reliability

- The system must maintain at least 99.9% uptime using cloud hosting.
- Regular data backups and recovery mechanisms must be in place.

Usability

- User Interface: Intuitive mobile-first design for Android devices.
- Accessibility: Follow accessibility guidelines (text readability, screen readers, etc.).

Security

- Authentication: Secure login with role-based access; optional multi-factor authentication.
- Data protection: Encrypt sensitive data in transit and at rest; implement Firestore security rules.

Compatibility

- Cross-platform support: Fully compatible with Android, with potential future iOS support.
- Backward Compatibility**:** Support at least two previous versions of Android OS.

Maintainability

- Code quality: Clean, modular code with documentation.
- Testing: Automated unit, integration, and end-to-end testing, along with manual usability testing.

Business Requirements

Business requirements define the broader goals CivicLink seeks to achieve, focusing on its value for citizens, officials, and local communities.

Community Engagement

- Citizen participation: Encourage active civic engagement through simplified reporting.
- Transparency: Real-time updates increase trust in local governance.

Efficiency & Accountability

- Streamline problem resolution and improve government response times.
- Provide analytics to officials for better decision-making and resource allocation.

Regulatory Compliance

- Data privacy: Ensure compliance with local data protection laws.
- Consent Management: Obtain user consent for data collection and sharing.

Technical Requirements

Technical requirements specify the architecture, frameworks, and tools necessary to develop and deploy CivicLink efficiently and securely.

Architecture

- Client-Server model: Flutter mobile app as the client; Firebase backend providing database, authentication, and notifications.
- Data Management: Cloud Firestore stores user profiles, issue reports, images, and interaction logs.
- Serverless Backend: Firebase Functions handle business logic for issue assignment, notifications, and analytics.

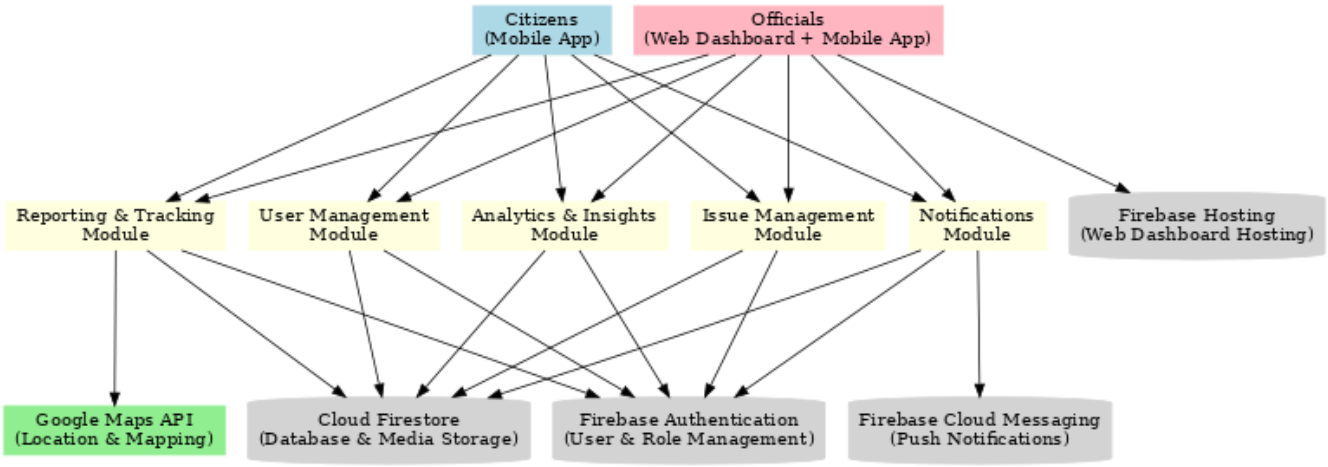
Development Environment

- Framework: Flutter (Dart) for cross-platform mobile development.
- Version Control: Git for code management, with separate branches for development, testing, and production.
- APIs: Google Maps API for location detection, issue mapping, and radius-based search.

ARCHITECTURE AND DESIGN

The architectural design of CivicLink is built on a robust and scalable client-server model that leverages modern cloud technologies to ensure reliability, security, and responsiveness. The system is designed to facilitate seamless interaction between citizens and local authorities, supporting real-time issue reporting, tracking, and resolution. The architecture integrates a Flutter-based cross-platform mobile application, a web dashboard for officials, and Firebase cloud services for backend operations, authentication, data management, and notifications. This design ensures a smooth user experience while maintaining transparency, accountability, and scalability to support growing community needs.

System architectural diagram



1. Front-End
   - User Interface (UI)
     - The mobile app is built using flutter, providing a responsive, intuitive, and mobile-first experience for citizens.
     - The official web dashboard is also integrated to allow administrators and officials to manage issues efficiently.
     - Flutter ensures consistent design and usability across devices.
   - User Roles
     - Citizens can report issues with photos and GPS-based location, track status, view nearby reports on a map, and manage their profile.
     - Officials can respond to issues, assign tasks, update statuses, and monitor performance analytics.
2. Firebase Authentication
   - User Registration and Login
     - Secure authentication is managed by Firebase Authentication, supporting role-based login for citizens and officials.

        - Session handling, password resets, and identity verification are included to maintain secure access.
    - Role-based Access control
        - Authentication processes determine feature access, ensuring citizens only access reporting features while officials gain access to management and analytics tools.

3. Firestore Database
    - Real-Time Data Synchronization
        - Cloud Firestore acts as the central database, storing issue reports, user details, photos, and status updates.
        - Its real-time sync ensures instant updates for both citizens and officials when an issue is reported, assigned, or resolved.
    - Data Structure
        - Issues, user profiles, comments, and status updates are organized in structured collections and documents for efficient querying and retrieval.

4. Firebase Cloud Messaging (FCM)
    - Push Notifications
        - FCM delivers instant notifications to citizens when an issue status changes (e.g., pending → in-progress → resolved).
        - Officials receive alerts about newly reported critical issues or pending requests, ensuring timely responses.

5. Firebase Cloud Storage
    - Media Handling
        - Firebase Storage is used for storing issue-related images submitted by citizens.
        - Provides secure, scalable storage with quick retrieval, ensuring smooth media display without affecting performance.

6. Google Maps API Integration
    - Location Services
        - The app automatically detects the user's location when reporting issues.
        - Citizens can view nearby issues on a map with an adjustable search radius and filters by department.
        - Officials can track the geographic distribution of issues for better resource allocation.

7. Analytics and Monitoring
    - Google Analytics for Firebase
        - Tracks user engagement, issue reporting trends, and feature utilization to improve the app.
    - Performance Monitoring
        - Monitors network latency, database queries, and app responsiveness to ensure smooth operation.

Interaction Flow

1. User Interaction
    - Citizens report issues through the Flutter mobile app by attaching a photo, description, and priority. The location is auto-detected via Google Maps API.
    - Officials access reported issues via the web dashboard or mobile app, reviewing details and responding accordingly.

2. Data Processing and Storage
    - Reported issues, media, and user data are securely stored in Firestore and Firebase Storage.
    - Authentication validates role-based access before granting permission.

3. Real-Time Updates
    - Firestore ensures that issue status changes are instantly reflected in the citizen's app.
    - Notifications are triggered by Firebase Cloud Messaging to keep both parties informed.

4. Security and Compliance
    - Firebase authentication and Firestore security rules enforce data privacy, preventing unauthorized access.
    - Data handling aligns with modern compliance standards for user protection.

5. Scalability and Performance
    - The serverless Firebase architecture automatically scales to handle increased users, reports, and media uploads without performance drops.

6. Analytics and Feedback
    - Insights from Google Analytics and Performance Monitoring help identify trends (e.g., frequently reported issues, response delays).

- This feedback loop enables continuous improvement and ensures local authorities can be more responsive.

METHODOLOGY

The development of CivicLink followed a systematic and iterative approach to ensure that the application was user-friendly, scalable, and secure. The methodology consisted of several stages, starting from requirement gathering to deployment and maintenance. Agile principles were adopted, allowing for incremental improvements through sprints and continuous feedback integration.

1. Requirement Analysis

The first stage involved gathering and analyzing requirements from both citizens and government officials to identify the most critical features. Potential users were interviewed to refine the scope and ensure the solution aligned with real-world needs. Some of the key features identified are;

- Citizen features: Issue reporting with automatic geolocation, photo upload, priority setting, issue status tracking, reminders, and issue map view.
- Official features: Issue assignment, updating statuses, responding to urgent issues, and performance tracking.
- Common features: User profiles, push notifications, app preferences, security, and help & support.

2. System Architecture Design

The system architecture was designed to be cloud-based, modular, and highly scalable, leveraging Firebase and Google Cloud services for backend functionality. System architecture components are listed below.

- Frontend: Built using Flutter, a cross-platform framework that ensures a responsive mobile experience. While the initial release targets Android devices, the architecture supports easy extension to iOS in future iterations.
- Backend: Firebase Authentication, Firestore (NoSQL database), and Firebase Cloud Storage for managing users, issues, and media.
- Cloud Services: Google Maps API for geolocation and map-based issue display; Firebase Cloud Functions for business logic like notifications and automated updates.

A detailed system architecture diagram was created to visualize the interactions between components and the flow of data across the system.

3. Technology Stack

The choice of technologies focused on scalability, real-time data handling, and ease of integration.

- Frontend
    - Framework: Flutter (Dart)
    - UI/UX: Material design principles with support for dark mode and multilingual options.
- Backend
    - Firebase Authentication: Secure user sign-in for citizens and officials
    - Firestore Database: Real-time NoSQL database for storing issues, profiles, and status updates
    - Firebase Cloud Storage: To handle media uploads such as photos of reported issues
    - Firebase Cloud Functions: For server-side automation, reminders, and notifications
- Cloud Services
    - Google Maps API: For mapping, geolocation, and search radius customization
    - Firebase Cloud Messaging (FCM): For real-time notifications

4. Frontend Development

The frontend was developed with a focus on usability and accessibility. Citizens and officials interact with tailored dashboards designed to streamline their tasks.

Development Process

1. UI/UX Design: Created wireframes and mockups using Figma, validated through user feedback.
2. Implementation: Built with Flutter's widget-based architecture, enabling dynamic UI and cross-platform support.
3. Testing: Conducted unit testing of screens and user acceptance testing (UAT) with sample users.

5. Backend Development

The backend utilized Firebase and Google Cloud to ensure secure authentication, fast data access, and minimal server management.

- Firestore Database: Stores citizen reports, status updates, and official responses with real-time synchronization.
- Firebase Authentication: Provides secure role-based login for both citizens and officials.

- Cloud Functions: Automates backend tasks such as sending notifications when issue statuses are updated.

6. Cloud Integration

Cloud services added essential features and enhanced app performance.

- Google Maps API: Enabled geolocation, search radius filtering, and map-based visualization of issues.
- Firebase Cloud Messaging (FCM): Sent push notifications to citizens and officials for issue updates.
- Firebase Analytics: Provided insights into user engagement and app performance.

7. Testing and Validation

Testing was carried out at different levels to ensure the system's functionality, reliability, and usability.

- Unit Testing: Verified individual components in both frontend and backend.
- Integration Testing: Ensured smooth communication between the frontend, Firebase, and cloud services.
- User Acceptance Testing (UAT): Feedback was collected from sample users to confirm usability and functionality.
- Performance Testing: Evaluated response times, real-time updates, and notification delivery under load.

8. Deployment and Maintenance

At this stage, the CivicLink application has been built as an Android APK, which is currently used for testing and demonstration purposes**.** Extending support to iOS devices is identified as a future enhancement.

Deployment steps

- Build: Generated a production-ready APK using Flutter's build tools for Android testing and evaluation.
- Future publishing: Plans are in place to publish the app to the Google Play Store, following store guidelines and compliance checks.
- Future iOS implementation: The cross-platform nature of Flutter will enable the development and deployment of an iOS version in later phases.
- Monitoring (Planned): Once published, Firebase Analytics and Crashlytics will be integrated to track performance, crashes, and user engagement.
- Feedback loop: A mechanism for users to provide feedback and suggest improvements will support continuous refinement post-deployment.

EVALUATION

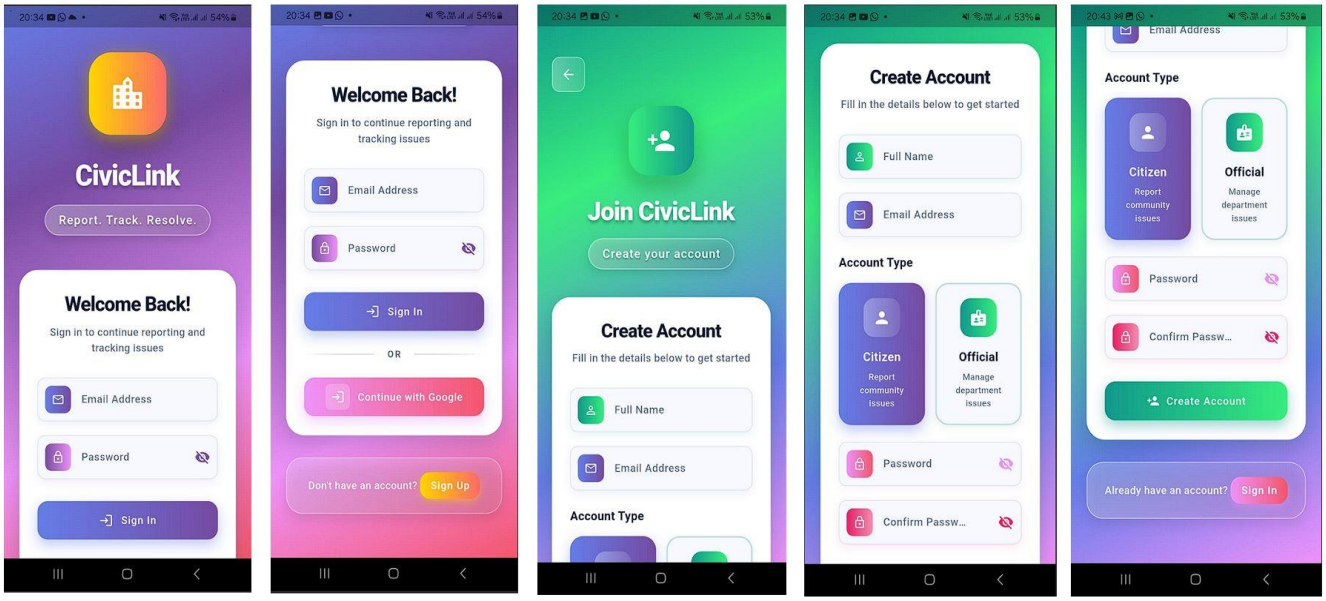


Figure 1:Login and Register Screens

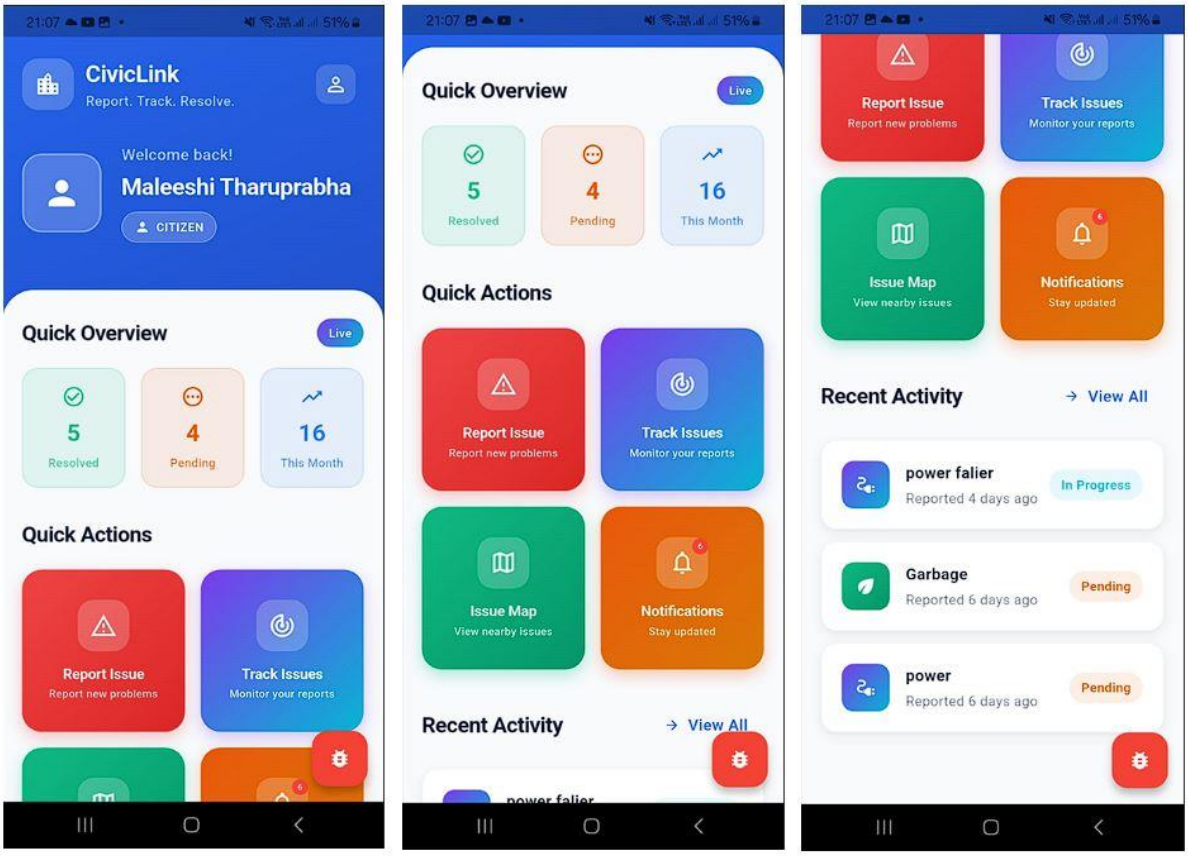


Figure 2: Home Page of citizen accounts

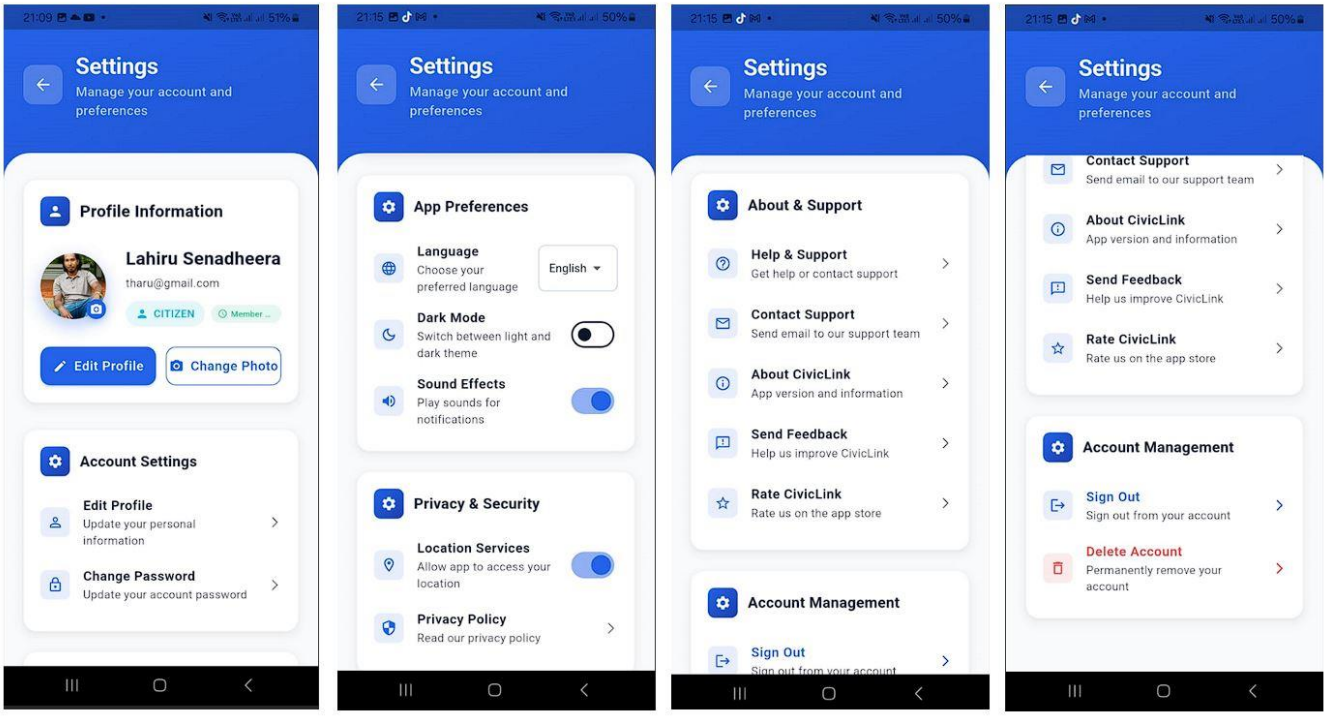


Figure 3: Settings Page of Citizen accounts

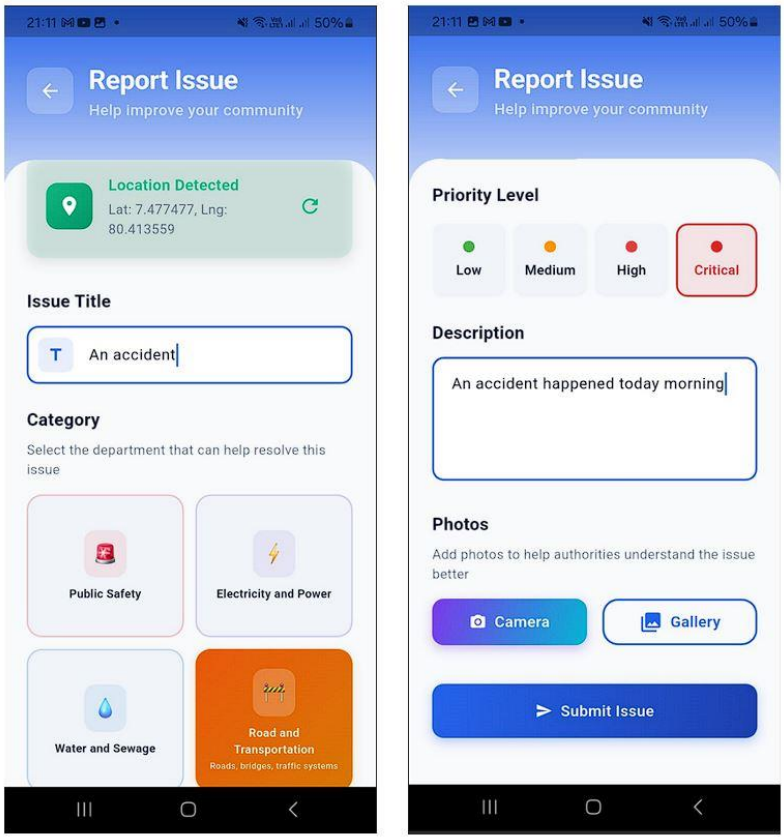


Figure 4:Report Issues Screen

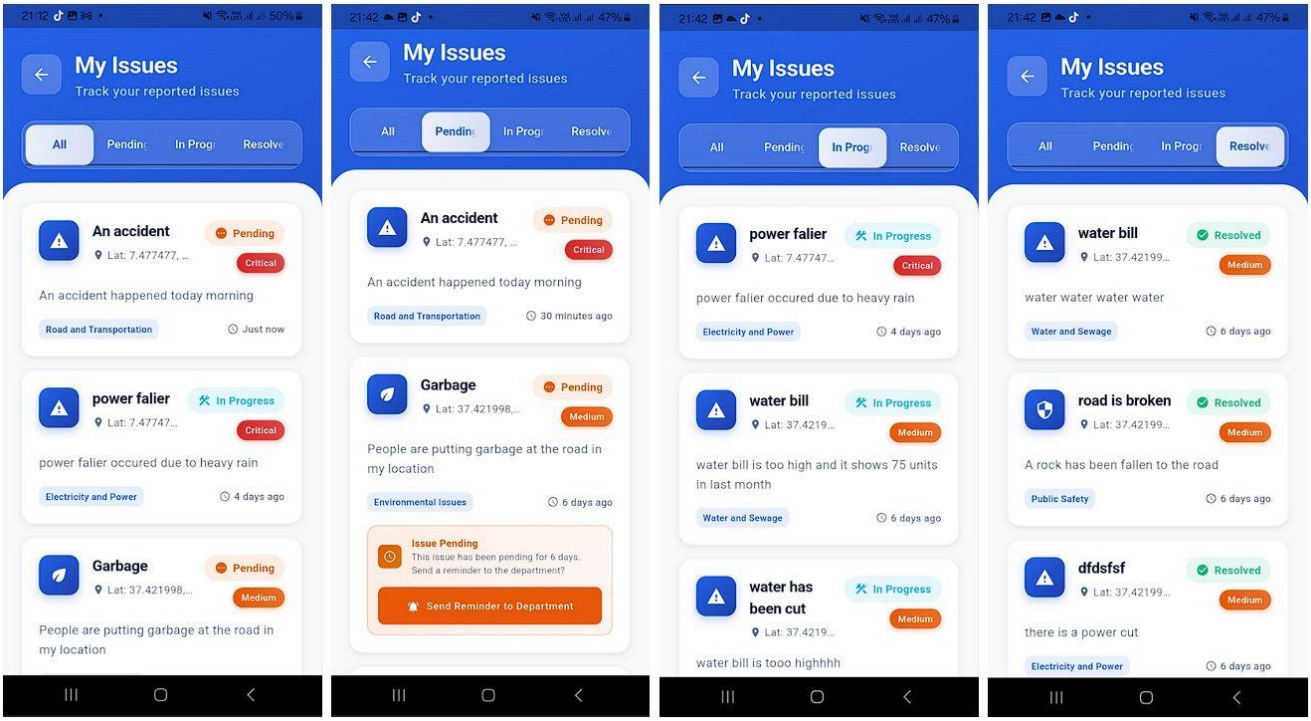


Figure 5:Track Issues Screen

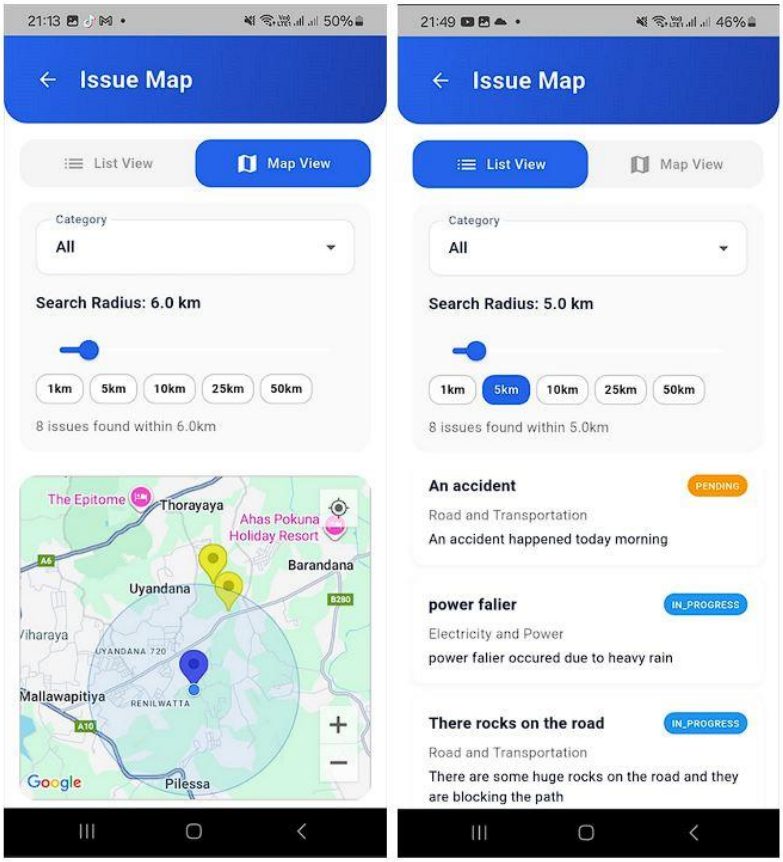
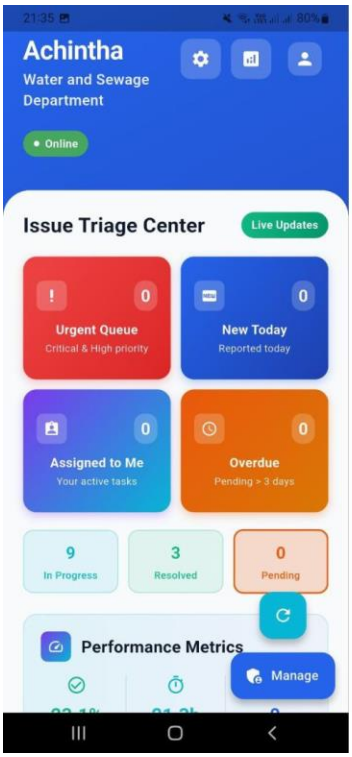
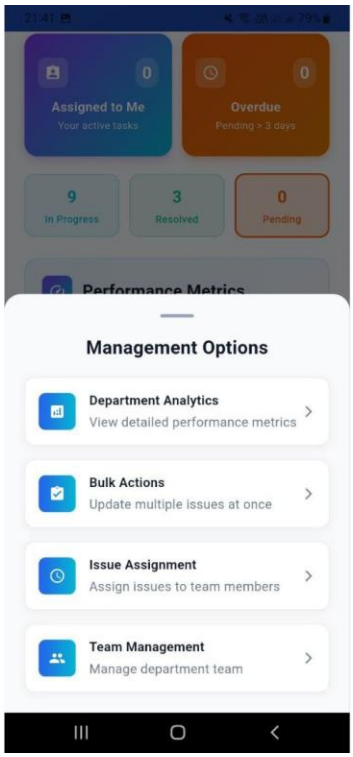


Figure 6:Issue Map screen



Figure 7:Notification screen





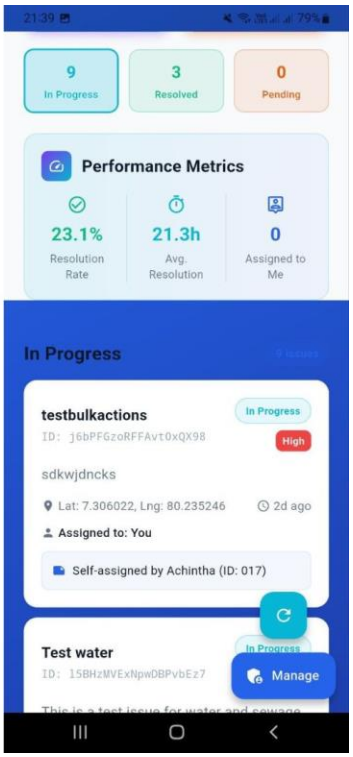


Figure 8 Department dashboard screenshots1

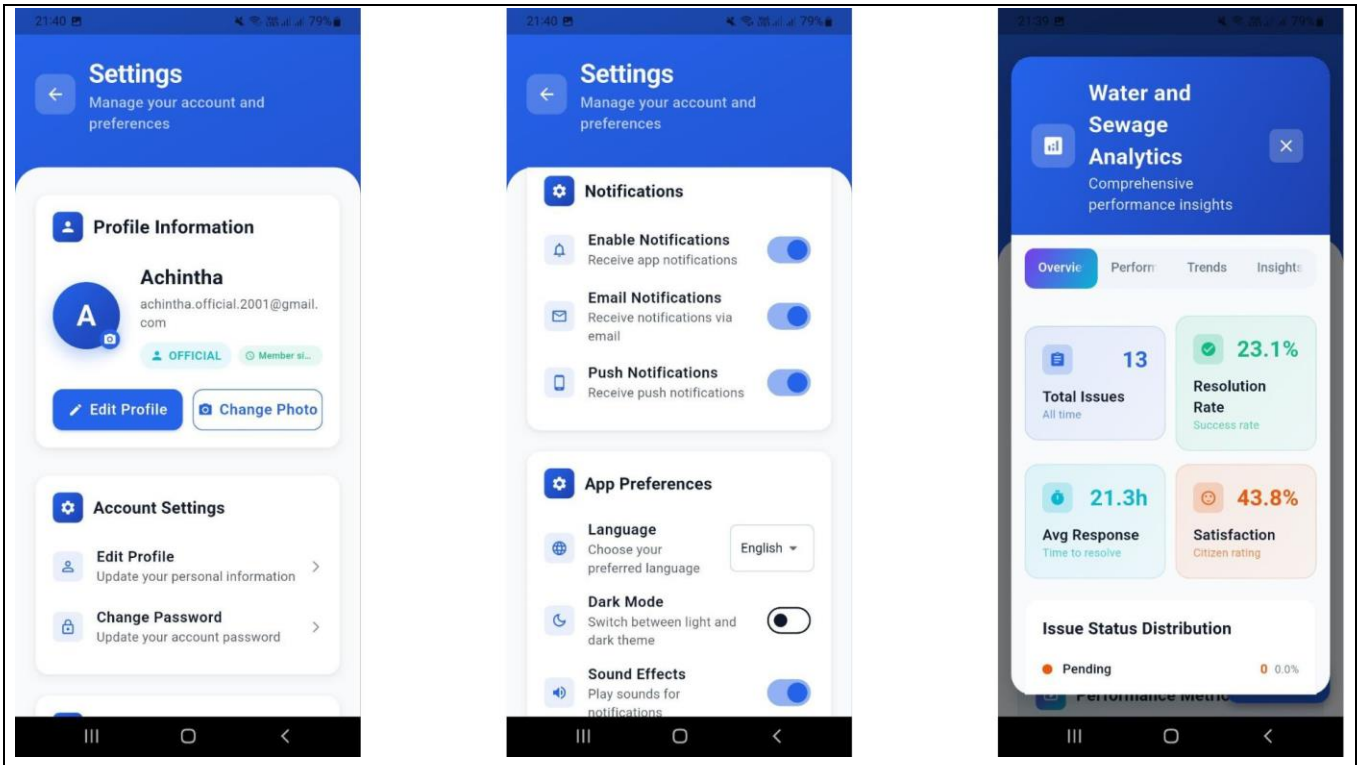


Figure 9 Department dashboard screenshot2

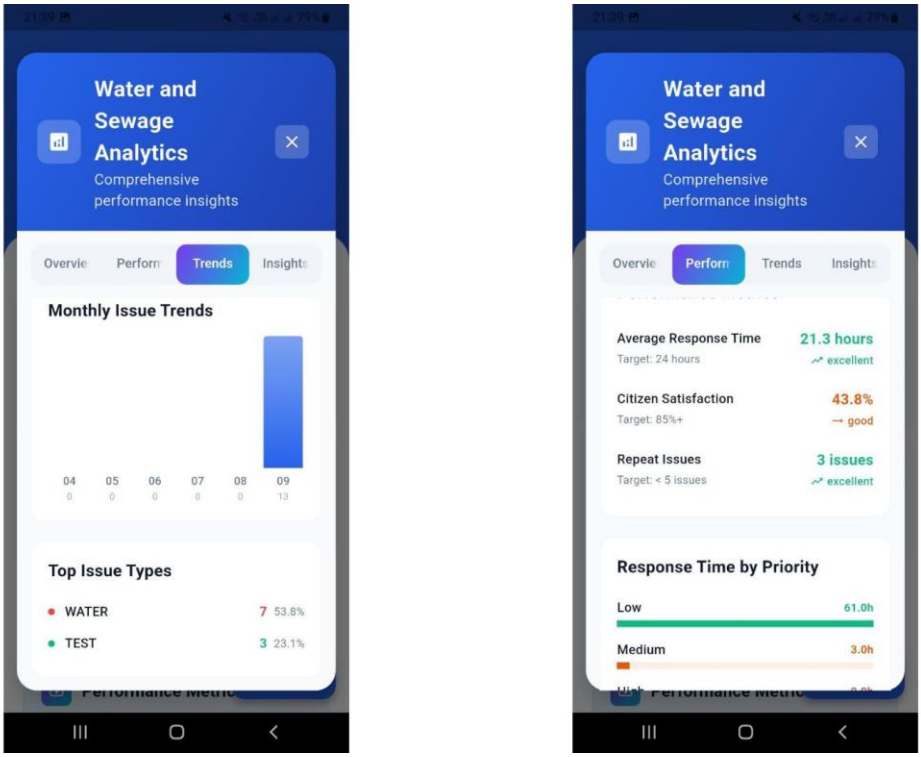


Figure 9 department dashboard screenshot3

REFERENCES

[1] “Flutter documentation,” *Flutter.dev*. [Online]. Available: https://docs.flutter.dev/. [Accessed: 12-Sept-2025].

[2] “Firebase documentation,” *Firebase*. [Online]. Available: https://firebase.google.com/docs. [Accessed: 12-Sept-2025].

[3] “Google maps platform documentation,” *Google for Developers*. [Online]. Available: https://developers.google.com/maps/documentation. [Accessed: 12-Sept-2025].